

Ouvrier Syntax Handbook

Version: v0.1

Scope: this handbook is exclusively about Ouvrier syntax and composition. It explains how to read and write Ouvrier declarations, from a single HTTP agent to complex agentic middleware with tools, policies, parallel work, maps, subagents, and non-HTTP triggers.

Table Of Contents

1. The Ouvrier grammar
2. Minimal program structure
3. Provider and model syntax
4. Pipeline layout rules
5. `Run`, `NewRunner`, and `Validate`
6. Trigger syntax with `From`
7. Agent step syntax with `Pipe`
8. Typed result syntax with `Output[T]`
9. Terminal syntax with `Reply`, `Push`, and `Sink`
10. Go tool syntax
11. Permission syntax
12. Sequential middleware composition
13. Parallel composition
14. Map composition
15. SubAgent composition
16. Skills, MCP, and Bash syntax
17. Runtime control syntax
18. Hooks, tracing, and state syntax
19. Complete keyword reference
20. Worked examples from simple to complex
21. Validation and troubleshooting checklist

1. The Ouvrier Grammar

The public Ouvrier mental model is:

```
From -> Pipe -> Pipe -> Reply
From -> Pipe -> Pipe -> Push
From -> Pipe -> Pipe -> Sink
```

An Ouvrier program declares agentic middleware. A trigger receives work, one or more pipes transform that work through LLM agent steps, and one terminal decides what happens to the final value.

The smallest useful syntax shape is:

```
err := ovr.Run(":8080",
    ovr.From("POST /route"),
    ovr.Pipe("Do one task.",
        ovr.Model("openai/gpt-5.1"),
    ),
    ovr.Reply(ovr.SSE()),
)
```

The most common production shape is:

```
type Result struct {
    Status string `json:"status"`
}

err := ovr.Run(":8080",
    ovr.From("POST /route"),
    ovr.Pipe("Do one task and return JSON.",
        ovr.Model("openai/gpt-5.1"),
        ovr.Output[Result](),
    ),
    ovr.Reply(ovr.JSON[Result]()),
)
```

The grammar is intentionally small:

```
program      = Run | NewRunner(...).Run
pipeline     = From, step*, terminal
step         = Pipe | Parallel | Map
terminal     = Reply | Push | Sink
childPipeline = Pipeline(Pipe, Pipe, ...)
pipeOption   = Model | Output | Tool | Skill | MCP | Bash | SubAgent |
              Timeout | MaxTokens | MaxCostUSD | Retry | NoCache |
              SequentialTools
```

Read Ouvrier code from left to right. Each node receives the previous node's output as its input. The final terminal consumes the last output.

2. Minimal Program Structure

Every manually written worker is a normal Go program.

```
package main

import (
    "log"

    ovr "github.com/ArnaudGuiovanna/ouvrier"
)
```

```
func main() {
    if err := ovr.Run(":8080",
        ovr.From("POST /test"),
        ovr.Pipe("Say hello.",
            ovr.Model("openai/gpt-5.1"),
        ),
        ovr.Reply(ovr.SSE()),
    ); err != nil {
        log.Fatal(err)
    }
}
```

Important points:

- `package main` makes an executable worker.
- The import alias is usually `ovr`.
- `ovr.Run` starts the default runtime.
- `:8080` is the listen address.
- All pipeline nodes are arguments after the address.
- Every `Pipe` must include `ovr.Model(...)`.
- Every top-level pipeline must end with exactly one terminal.

3. Provider And Model Syntax

Model IDs use the form:

```
provider/model-name
```

Examples:

```
ovr.Model("openai/gpt-5.1")
ovr.Model("anthropic/claude-sonnet-4-6")
ovr.Model("ollama/llama3.1")
ovr.Model("mistral/mistral-large-latest")
ovr.Model("gemini/gemini-2.0-flash")
ovr.Model("vllm/qwen2.5-coder")
```

Supported provider prefixes:

Prefix	Environment variables
anthropic/	ANTHROPIC_API_KEY , optional ANTHROPIC_BASE_URL
openai/	OPENAI_API_KEY , optional OPENAI_BASE_URL
ollama/	optional OLLAMA_BASE_URL
mistral/	MISTRAL_API_KEY , optional MISTRAL_BASE_URL
gemini/	GEMINI_API_KEY , optional GEMINI_BASE_URL
vllm/	optional VLLM_API_KEY , VLLM_BASE_URL

The model prefix must match the configured provider. If the code says:

```
ovr.Model("anthropic/claude-sonnet-4-6")
```

then setting only `OPENAI_API_KEY` does not configure that pipe. Use:

```
ovr.Model("openai/gpt-5.1")
```

for OpenAI.

Ouvrier reads provider credentials from the process environment. A `.env` file is only useful if your shell or process manager loads it before the worker starts.

```
set -a
source .env
set +a
go run .
```

Fail fast when a worker needs a provider key:

```
func main() {
    if err := ovr.RequireEnv("OPENAI_API_KEY"); err != nil {
        log.Fatal(err)
    }

    if err := ovr.Run(":8080",
        ovr.From("POST /test"),
        ovr.Pipe("Answer the request.", ovr.Model("openai/gpt-5.1")),
        ovr.Reply(ovr.SSE()),
    ); err != nil {
        log.Fatal(err)
    }
}
```

4. Pipeline Layout Rules

Ouvrier validates the declaration before serving traffic.

Valid:

```
ovr.Run(":8080",
    ovr.From("POST /tickets"),
    ovr.Pipe("Classify the ticket.", ovr.Model("openai/gpt-5.1")),
    ovr.Reply(ovr.SSE()),
)
```

Invalid: the first node is not `From`.

```
ovr.Run(":8080",
    ovr.Pipe("Classify.", ovr.Model("openai/gpt-5.1")),
    ovr.Reply(ovr.SSE()),
)
```

Invalid: the pipe has no model.

```
ovr.Run(":8080",
    ovr.From("POST /tickets"),
    ovr.Pipe("Classify."),
    ovr.Reply(ovr.SSE()),
)
```

Invalid: the terminal is not last.

```
ovr.Run(":8080",
    ovr.From("POST /tickets"),
    ovr.Reply(ovr.SSE()),
    ovr.Pipe("This node is unreachable.", ovr.Model("openai/gpt-5.1")),
)
```

Invalid: `Reply` requires an HTTP trigger.

```
ovr.Run(":8080",
    ovr.From(ovr.Cron("@every 1h")),
    ovr.Pipe("Run scheduled work.", ovr.Model("openai/gpt-5.1")),
    ovr.Reply(ovr.SSE()),
)
```

For cron and stream triggers, use `Push(...)` or `Sink(...)`.

Multiple top-level pipelines are allowed. Each new pipeline starts with its own `From`.

```
ovr.Run(":8080",
    ovr.From("POST /tickets"),
    ovr.Pipe("Classify tickets.", ovr.Model("openai/gpt-5.1")),
    ovr.Reply(ovr.SSE()),

    ovr.From("GET /health-text"),
    ovr.Pipe("Return a short health sentence.", ovr.Model("openai/gpt-5.1")),
    ovr.Reply(ovr.SSE()),
)
```

5. Run, NewRunner, And Validate

Use `ovr.Run` for the default runtime:

```
err := ovr.Run(":8080", nodes...)
```

Use `ovr.NewRunner` when the syntax needs advanced runtime configuration:

```
runner := ovr.NewRunner(
    ovr.WithSchemaRepairAttempts(1),
    ovr.WithPermissionPolicy(ovr.AllowSideEffects("email")),
)

err := runner.Run(":8080", nodes...)
```

Use `ovr.Validate` to check a pipeline without starting the server:

```
if err := ovr.Validate(  
    ovr.From("POST /tickets"),  
    ovr.Pipe("Classify.", ovr.Model("openai/gpt-5.1")),  
    ovr.Reply(ovr.SSE()),  
); err != nil {  
    log.Fatal(err)  
}
```

Runner options:

Syntax	Meaning
<code>ovr.WithPermissionPolicy(policy)</code>	Installs permission rules for side effects.
<code>ovr.WithStateStore(store)</code>	Uses a custom durable state store.
<code>ovr.WithHooks(hooks)</code>	Installs lifecycle event hooks.
<code>ovr.WithSandbox(ovr.Sandbox(root))</code>	Sets a filesystem boundary for file sinks and sandboxed capabilities.
<code>ovr.WithSchemaRepairAttempts(n)</code>	Allows bounded repair attempts when typed output is invalid.
<code>ovr.WithTracer(tracer)</code>	Emits OTel-compatible spans through a tracer adapter.

6. Trigger Syntax With From

`From` declares how work enters a pipeline.

```
ovr.From(source, options...)
```

Sources:

Syntax	Meaning
<code>ovr.From("GET /path")</code>	HTTP GET route.
<code>ovr.From("POST /path")</code>	HTTP POST route.
<code>ovr.From(ovr.Cron("0 6 * * *"))</code>	Cron trigger.
<code>ovr.From(ovr.Cron("@every 10s"))</code>	Interval trigger.
<code>ovr.From(ovr.Webhook("github"))</code>	Inbound webhook route at <code>/webhooks/github</code> .
<code>ovr.From(ovr.Stream("nats://host:4222/subject"))</code>	Stream receiver.

From options:

Syntax	Meaning
<code>ovr.WorkerPool(20)</code>	Limits concurrent executions for this trigger.
<code>ovr.IdempotencyKey("Idempotency-Key")</code>	Uses a header to avoid duplicate trigger deliveries.
<code>ovr.VerifySignature("SECRET_ENV", "X-Signature")</code>	Verifies an HMAC-SHA256 signature before execution.

HTTP Triggers

HTTP route strings must be:

```
METHOD /path
```

Supported methods are `GET` and `POST`.

```
ovr.From("GET /health")
ovr.From("POST /tickets")
ovr.From("POST /tickets/{id}")
```

Path parameters use braces:

```
ovr.From("POST /tickets/{id}")
```

If a route has no path parameters, the request body is passed to the first pipe as-is. If a route has path parameters, the first pipe receives JSON like:

```
{
  "path_params": {
    "id": "T-123"
  },
  "body": {
    "message": "user cannot sign in"
  }
}
```

Cron Triggers

Cron supports five-field expressions:

```
ovr.From(ovr.Cron("0 6 * * *"))
ovr.From(ovr.Cron("*/15 9-17 * * 1-5"))
```

It also supports interval syntax:

```
ovr.From(ovr.Cron("@every 1h"))
ovr.From(ovr.Cron("@every 10s"))
```

Cron pipelines cannot use `Reply`. Use `Push` or `Sink`.

```
ovr.Run(":8080",
  ovr.From(ovr.Cron("@every 1h")),
```

```
    ovr.Pipe("Summarize scheduled work.", ovr.Model("openai/gpt-5.1")),
    ovr.Sink(ovr.Log()),
)
```

Webhook Triggers

Webhook names may contain letters, digits, dot, dash, and underscore.

```
ovr.From(ovr.Webhook("github"))
ovr.From(ovr.Webhook("stripe-events"))
```

The runtime exposes:

```
POST /webhooks/<provider>
```

The first pipe receives JSON like:

```
{
  "trigger": "webhook",
  "provider": "github",
  "body": {
    "action": "opened"
  }
}
```

Signed webhook syntax:

```
ovr.From(ovr.Webhook("github"),
    ovr.VerifySignature("GITHUB_WEBHOOK_SECRET", "X-Hub-Signature-256"),
    ovr.IdempotencyKey("X-GitHub-Delivery"),
)
```

Stream Triggers

Stream sources use URIs.

```
ovr.From(ovr.Stream("nats://127.0.0.1:4222/tickets.created"))
ovr.From(ovr.Stream("redis://127.0.0.1:6379/tickets"))
ovr.From(ovr.Stream("kafka://localhost:9092/tickets"))
```

Stream inputs are wrapped for the first pipe:

```
{
  "trigger": "stream",
  "uri": "nats://127.0.0.1:4222/tickets.created",
  "id": "message-id",
  "body": {
    "ticket_id": "T-123"
  },
  "metadata": {
    "source": "support"
  }
}
```

Stream pipelines cannot use `Reply`. Use `Push` or `Sink`.

7. Agent Step Syntax With Pipe

`Pipe` declares one agent step.

```
ovr.Pipe(goal string, options ...ovr.PipeOption)
```

The goal is the natural-language instruction for that step:

```
ovr.Pipe("Summarize the request in one sentence.",
        ovr.Model("openai/gpt-5.1"),
    )
```

Pipe options:

Syntax	Meaning
<code>ovr.Model(id)</code>	Required. Selects provider and model.
<code>ovr.Output[T]()</code>	Requires final pipe output to be JSON matching <code>T</code> .
<code>ovr.Tool(name, fn, opts...)</code>	Exposes a Go function as a tool.
<code>ovr.Skill(name)</code>	Loads <code>skills/<name>/SKILL.md</code> into the pipe context.
<code>ovr.MCP(name)</code>	Connects a configured MCP server to the pipe.
<code>ovr.Bash(sandbox, opts...)</code>	Exposes a sandboxed Bash tool.
<code>ovr.SubAgent(name, pipeline, opts...)</code>	Exposes a child pipeline as a governed tool.
<code>ovr.Timeout("30s")</code>	Sets a wall-clock budget for the pipe.
<code>ovr.MaxTokens(20000)</code>	Sets a token budget.
<code>ovr.MaxCostUSD(2.0)</code>	Sets a cost budget in US dollars.
<code>ovr.Retry(2, ovr.ExponentialBackoff())</code>	Retries transient provider failures and retry-safe tools.
<code>ovr.NoCache()</code>	Disables provider prompt-cache hints for this pipe.
<code>ovr.SequentialTools()</code>	Runs tool calls from one model turn one at a time.

Good pipe goals are specific about output shape and task boundaries:

```
ovr.Pipe("Extract the customer email and ticket category. Return JSON only.",
        ovr.Model("openai/gpt-5.1"),
        ovr.Output[TicketFacts](),
    )
```

Avoid vague goals:

```
ovr.Pipe("Do it.", ovr.Model("openai/gpt-5.1"))
```

8. Typed Result Syntax With `Output[T]`

`Output[T]()` asks Ouvrier to generate a strict JSON Schema from Go type `T`, instruct the model to produce that shape, validate the result, and record schema violations.

```
type Triage struct {
    Priority string `json:"priority"`
    Summary  string `json:"summary"`
}

ovr.Pipe("Triage the ticket.",
    ovr.Model("openai/gpt-5.1"),
    ovr.Output[Triage](),
)
```

Use exported fields with JSON tags:

```
type Good struct {
    Status string `json:"status"`
}
```

Avoid misspelled tags:

```
type Bad struct {
    Status string `json:"statu"`
}
```

If a `Reply(JSON[T]())` is present and the last pipe has no `Output[T]()`, Ouvrier applies the reply schema to the last pipe.

```
ovr.Pipe("Return a status object.", ovr.Model("openai/gpt-5.1")),
ovr.Reply(ovr.JSON[Status]()),
```

If both are present, they must match:

```
ovr.Pipe("Return a status object.",
    ovr.Model("openai/gpt-5.1"),
    ovr.Output[Status](),
),
ovr.Reply(ovr.JSON[Status]()),
```

This is invalid because the pipe and reply schemas differ:

```
ovr.Pipe("Return a status object.",
    ovr.Model("openai/gpt-5.1"),
    ovr.Output[Status](),
),
ovr.Reply(ovr.JSON[DifferentStatus]()),
```

Multiple pipes may each have `Output[T]()`:

```

ovr.Pipe("Extract facts.",
    ovr.Model("openai/gpt-5.1"),
    ovr.Output[Facts](),
),
ovr.Pipe("Write a customer reply from those facts.",
    ovr.Model("openai/gpt-5.1"),
    ovr.Output[ReplyBody](),
),
ovr.Reply(ovr.JSON[ReplyBody]()),

```

Only the final terminal response is sent to the HTTP caller. Intermediate outputs are used as inputs to later steps.

Enable bounded repair attempts with a runner:

```
runner := ovr.NewRunner(ovr.WithSchemaRepairAttempts(1))
```

9. Terminal Syntax With Reply, Push, And Sink

Every top-level pipeline needs exactly one terminal.

Reply

Reply answers the HTTP caller.

```

ovr.Reply(ovr.JSON[Result]())
ovr.Reply(ovr.SSE())
ovr.Reply(ovr.Accepted())

```

Use `JSON[T]()` for typed JSON:

```

type Result struct {
    Status string `json:"status"`
}

ovr.Reply(ovr.JSON[Result]())

```

Use `SSE()` for server-sent events:

```
ovr.Reply(ovr.SSE())
```

Use `Accepted()` when the HTTP request should return `202 Accepted` while the pipeline continues asynchronously:

```
ovr.Reply(ovr.Accepted())
```

Reply is only compatible with HTTP triggers.

Push

`Push` sends the final output somewhere else.

```
ovr.Push(ovr.Webhook("https://example.com/results"))
ovr.Push(ovr.Queue("https://queue.example.com/jobs"))
ovr.Push(ovr.Queue("nats://127.0.0.1:4222/results"))
```

`Webhook(...)` has two meanings depending on context:

```
ovr.From(ovr.Webhook("github"))           // inbound trigger
ovr.Push(ovr.Webhook("https://example.com/out")) // outbound push target
```

Outbound push is a side effect. It needs target-scoped permission:

```
runner := ovr.NewRunner(
    ovr.WithPermissionPolicy(
        ovr.AllowSideEffectTargets("webhook", "https://example.com/results"),
    ),
)
```

Queue push also needs target-scoped permission:

```
runner := ovr.NewRunner(
    ovr.WithPermissionPolicy(
        ovr.AllowSideEffectTargets("queue", "nats://127.0.0.1:4222/results"),
    ),
)
```

Sink

`Sink` consumes the final output without replying to the trigger.

```
ovr.Sink(ovr.Log())
ovr.Sink(ovr.File("result.json"))
```

`Log()` is read-only from the permission-policy point of view.

`File(path)` writes to disk and requires:

- a runner sandbox
- target-scoped file permission

```
runner := ovr.NewRunner(
    ovr.WithSandbox(ovr.Sandbox("./out")),
    ovr.WithPermissionPolicy(
        ovr.AllowSideEffectTargets("file", "result.json"),
    ),
)

err := runner.Run(":8080",
    ovr.From("POST /reports"),
    ovr.Pipe("Generate a report.", ovr.Model("openai/gpt-5.1")),
```

```
) ovr.Sink(ovr.File("result.json")),
```

10. Go Tool Syntax

Tools are normal Go functions that the agent can call.

```
func LookupTicket(ctx context.Context, args struct {
    ID string `json:"id"`
}) (Ticket, error) {
    return Ticket{ID: args.ID}, nil
}
```

Register the function on a pipe:

```
ovr.Tool("lookup_ticket", LookupTicket,
    ovr.ReadOnly(),
    ovr.Describe("Load one ticket by ID."),
    ovr.Param("id", "Ticket identifier."),
    ovr.ToolTimeout("5s"),
)
```

Tool signature rules:

Rule	Valid syntax
First parameter must be <code>context.Context</code> .	<code>func(context.Context) error</code>
Optional second parameter is the JSON argument object.	<code>func(context.Context, Args) error</code>
Return either <code>error</code> or <code>(value, error)</code> .	<code>func(context.Context, Args) (Result, error)</code>
Arguments and results should be JSON-serializable.	Structs with JSON tags.

Valid:

```
func Ping(ctx context.Context) error {
    return nil
}

func Lookup(ctx context.Context, args LookupArgs) (Ticket, error) {
    return Ticket{}, nil
}
```

Invalid:

```
func MissingContext(args LookupArgs) (Ticket, error) {
    return Ticket{}, nil
}

func MissingError(ctx context.Context) string {
    return "ok"
}
```

Tool options:

Syntax	Meaning
<code>ovr.ReadOnly()</code>	The tool has no side effects.
<code>ovr.Idempotent("ticket.id")</code>	The tool mutates state but is replay-safe for a stable key.
<code>ovr.SideEffecting("email")</code>	The tool mutates state and needs permission.
<code>ovr.RequiresApproval()</code>	The tool is blocked unless policy explicitly allows it.
<code>ovr.Describe(text)</code>	Description shown to the model.
<code>ovr.Param(name, description)</code>	Parameter description shown to the model.
<code>ovr.ToolTimeout("10s")</code>	Per-tool wall-clock limit.

Read-Only Tool

```
type Ticket struct {
    ID      string `json:"id"`
    Subject string `json:"subject"`
    Body    string `json:"body"`
}

type LoadTicketArgs struct {
    ID string `json:"id"`
}

func LoadTicket(ctx context.Context, args LoadTicketArgs) (Ticket, error) {
    return Ticket{
        ID:      args.ID,
        Subject: "Login issue",
        Body:    "User cannot sign in.",
    }, nil
}

ovr.Pipe("Triage the ticket.",
    ovr.Model("openai/gpt-5.1"),
    ovr.Tool("load_ticket", LoadTicket,
        ovr.ReadOnly(),
        ovr.Describe("Load one support ticket by ID."),
        ovr.Param("id", "Ticket identifier."),
    ),
    ovr.Output[Triage](),
)
```

Side-Effecting Tool

A model cannot write files, send email, call your database, or mutate external state unless you give it a tool. Natural language alone is not a capability.

```
type SaveMarkdownArgs struct {
    Filename string `json:"filename"`
    Body     string `json:"body"`
}

type SavedFile struct {
```

```

    Path string `json:"path"`
}

func SaveMarkdown(ctx context.Context, args SaveMarkdownArgs) (SavedFile, error) {
    name := filepath.Base(args.Filename)
    if name == "." || name == string(filepath.Separator) {
        name = "result.md"
    }
    if filepath.Ext(name) != ".md" {
        name += ".md"
    }

    dir := "out"
    if err := os.MkdirAll(dir, 0o755); err != nil {
        return SavedFile{}, err
    }

    path := filepath.Join(dir, name)
    if err := os.WriteFile(path, []byte(args.Body), 0o600); err != nil {
        return SavedFile{}, err
    }
    return SavedFile{Path: path}, nil
}

```

Register and allow it:

```

runner := ovr.NewRunner(
    ovr.WithPermissionPolicy(ovr.AllowSideEffects("filesystem")),
)

err := runner.Run(":8080",
    ovr.From("POST /haiku"),
    ovr.Pipe("Write an eight-word haiku.",
        ovr.Model("openai/gpt-5.1"),
    ),
    ovr.Pipe("Save the previous haiku as Markdown and return the saved path.",
        ovr.Model("openai/gpt-5.1"),
        ovr.Tool("save_markdown", SaveMarkdown,
            ovr.SideEffecting("filesystem"),
            ovr.Describe("Save Markdown content to a local .md file."),
            ovr.Param("filename", "Base file name to create."),
            ovr.Param("body", "Markdown content to write."),
        ),
        ovr.Output[SavedFile](),
    ),
    ovr.Reply(ovr.JSON[SavedFile]()),
)

```

11. Permission Syntax

Ouvrier separates declaring a capability from allowing it.

Declaring a side-effecting tool:

```

ovr.Tool("send_email", SendEmail,
    ovr.SideEffecting("email"),
)

```

Allowing that label:

```
runner := ovr.NewRunner(  
    ovr.WithPermissionPolicy(ovr.AllowSideEffects("email")),  
)
```

Target-scoped actions use `AllowSideEffectTargets`:

```
ovr.AllowSideEffectTargets("webhook", "https://example.com/results")  
ovr.AllowSideEffectTargets("queue", "nats://127.0.0.1:4222/results")  
ovr.AllowSideEffectTargets("file", "result.json")  
ovr.AllowSideEffectTargets("process", "")  
ovr.AllowSideEffectTargets("filesystem", "")  
ovr.AllowSideEffectTargets("mcp:moodle-mcp", "moodle-mcp")
```

Use `AllowSideEffects` for non-targeted Go tool labels. Use `AllowSideEffectTargets` for output actions, Bash, MCP, and file sinks.

Permission syntax does not sanitize your tool implementation. A file-writing tool should still constrain paths. An email tool should still validate recipients. A database tool should still enforce domain permissions.

12. Sequential Middleware Composition

Multiple `Pipe` nodes compose sequentially.

```
type Facts struct {  
    CustomerEmail string `json:"customer_email"`  
    Category      string `json:"category"`  
}  
  
type CustomerReply struct {  
    Subject string `json:"subject"`  
    Body    string `json:"body"`  
}  
  
err := ovr.Run(":8080",  
    ovr.From("POST /tickets"),  
    ovr.Pipe("Extract customer email and category from the request.",  
        ovr.Model("openai/gpt-5.1"),  
        ovr.Output[Facts](),  
    ),  
    ovr.Pipe("Write a helpful support reply from the extracted facts.",  
        ovr.Model("openai/gpt-5.1"),  
        ovr.Output[CustomerReply](),  
    ),  
    ovr.Reply(ovr.JSON[CustomerReply]()),  
)
```

The first pipe receives the trigger input. The second pipe receives the first pipe's output. The reply receives the second pipe's output.

Use sequential pipes when each step depends on the previous step:

- normalize input

- extract facts
- enrich with tools
- draft final output
- validate or rewrite final output

Do not add a pipe just to make the code longer. A pipe boundary should mean a real task boundary.

13. Parallel Composition

`Parallel` fans out the same input to multiple pipe branches.

```
type ReviewSet struct {
    Results []string `json:"results"`
}

err := ovr.Run(":8080",
    ovr.From("POST /drafts"),
    ovr.Parallel(
        ovr.Pipe("Check factual quality.",
            ovr.Model("openai/gpt-5.1"),
        ),
        ovr.Pipe("Check policy compliance.",
            ovr.Model("openai/gpt-5.1"),
        ),
    ),
    ovr.Pipe("Combine the review results into one JSON answer.",
        ovr.Model("openai/gpt-5.1"),
        ovr.Output[ReviewSet](),
    ),
    ovr.Reply(ovr.JSON[ReviewSet]()),
)
```

Rules:

- Every item inside `Parallel(...)` must be a `Pipe`.
- Each branch receives the same input.
- Results are returned in declaration order.
- Without `PartialOK()`, one branch failure fails the whole step.
- With `PartialOK()`, failed branches become ordered error outcomes.

```
ovr.Parallel(
    ovr.Pipe("Check factual quality.", ovr.Model("openai/gpt-5.1")),
    ovr.Pipe("Check policy compliance.", ovr.Model("openai/gpt-5.1")),
    ovr.PartialOK(),
)
```

Use `Parallel` when independent checks or transformations can run at the same time.

14. Map Composition

`Map` applies one or more pipe steps to each element of the previous JSON array.

```
type TicketItem struct {
    ID    string `json:"id"`
    Text  string `json:"text"`
}

type TicketScore struct {
    ID        string `json:"id"`
    Priority  string `json:"priority"`
}

type BatchReply struct {
    Scores []TicketScore `json:"scores"`
}

err := ovr.Run(":8080",
    ovr.From("POST /batch"),
    ovr.Pipe("Extract a JSON array of ticket items from the request.",
        ovr.Model("openai/gpt-5.1"),
        ovr.Output[[]TicketItem](),
    ),
    ovr.Map(
        ovr.Concurrency(4),
        ovr.Pipe("Score one ticket item.",
            ovr.Model("openai/gpt-5.1"),
            ovr.Output[TicketScore](),
        ),
    ),
    ovr.Pipe("Wrap the ordered scores in the final reply object.",
        ovr.Model("openai/gpt-5.1"),
        ovr.Output[BatchReply](),
    ),
    ovr.Reply(ovr.JSON[BatchReply]()),
)
```

Rules:

- The input to `Map` must be a JSON array.
- Every node inside `Map(...)` must be a `Pipe`.
- `Concurrency(limit)` bounds concurrent item executions.
- `Concurrency(0)` is invalid.
- `PartialOK()` returns ordered partial outcomes instead of failing the whole map immediately.

```
ovr.Map(
    ovr.Concurrency(8),
    ovr.PartialOK(),
    ovr.Pipe("Score one item.", ovr.Model("openai/gpt-5.1")),
)
```

Use `Map` for batches: many tickets, many records, many messages, many rows.

15. SubAgent Composition

A `SubAgent` exposes a child pipeline as a governed tool to a parent pipe.

First declare a child pipeline:

```
type Translation struct {
    Text string `json:"text"`
}

translator := ovr.Pipeline(
    ovr.Pipe("Translate the input to French.",
        ovr.Model("openai/gpt-5.1"),
        ovr.Output[Translation](),
    ),
)
```

Then attach it to a parent pipe:

```
type FinalReply struct {
    English string `json:"english"`
    French  string `json:"french"`
}

err := ovr.Run(":8080",
    ovr.From("POST /reply"),
    ovr.Pipe("Draft a bilingual reply. Use the translate subagent when needed.",
        ovr.Model("openai/gpt-5.1"),
        ovr.SubAgent("translate", translator, ovr.MaxParallel(2)),
        ovr.Output[FinalReply](),
    ),
    ovr.Reply(ovr.JSON[FinalReply]()),
)
```

Rules:

- `Pipeline(...)` for a subagent may contain only `Pipe` nodes.
- `SubAgent(name, pipeline, ...)` is registered on a parent `Pipe`.
- The parent model decides when to call the subagent tool.
- Child sessions inherit governance, cancellation, and budgets.
- `MaxParallel(limit)` must be between 1 and 5.
- `PartialOK()` lets subagent task failures return ordered error outcomes.
- Recursive subagent graphs are rejected.

```
ovr.SubAgent("translate", translator,
    ovr.MaxParallel(2),
    ovr.PartialOK(),
)
```

Use subagents when a reusable child workflow is more than a single Go tool:

- translate then validate
- research then summarize

- inspect a batch then score
- draft then critique

16. Skills, MCP, And Bash Syntax

Skills

`skill` loads reusable Markdown instructions from a skill directory.

```
ovr.Pipe("Triage the ticket.",
    ovr.Model("openai/gpt-5.1"),
    ovr.Skill("ticket-triage"),
)
```

Rules:

- The name must be a direct child of `skills`.
- It cannot be `.` or `...`.
- It cannot contain path separators.

MCP

`MCP` connects a configured Model Context Protocol server to a pipe.

```
ovr.Pipe("Use Moodle tools to inspect learner progress.",
    ovr.Model("openai/gpt-5.1"),
    ovr.MCP("moodle-mcp"),
)
```

Rules:

- The MCP server name cannot be empty.
- It cannot be `.` or `...`.
- It cannot contain path separators.
- MCP side effects need target-scoped permission, commonly
`ovr.AllowSideEffectTargets("mcp:<server-name>", "<server-name>").`

Bash

`Bash` exposes a sandboxed command tool named `bash`.

```
runner := ovr.NewRunner(
    ovr.WithPermissionPolicy(
        ovr.AllowSideEffectTargets("process", "*"),
        ovr.AllowSideEffectTargets("filesystem", "*"),
    ),
)

err := runner.Run(":8080",
    ovr.From("POST /inspect"),
```

```

    ovr.Pipe("Inspect files in the workspace.",
        ovr.Model("openai/gpt-5.1"),
        ovr.Bash(ovr.Sandbox("./workspace", ovr.AllowEnv("PATH")),
            ovr.BashTimeout("10s"),
            ovr.BashMaxOutputBytes(32*1024),
        ),
    ),
    ovr.Reply(ovr.SSE()),
)

```

Bash options:

Syntax	Meaning
<code>ovr.BashTimeout("10s")</code>	Per-command timeout.
<code>ovr.BashMaxOutputBytes(32768)</code>	Captured stdout/stderr bound.
<code>ovr.UnsafeBashHostExecution()</code>	Allows host-shell fallback. Use only for local or trusted workloads.

Sandbox syntax:

```

ovr.Sandbox("./workspace")
ovr.Sandbox("./workspace", ovr.AllowEnv("PATH", "HOME"))

```

17. Runtime Control Syntax

Budgets

Pipe budgets are declared as pipe options:

```

ovr.Pipe("Summarize the document.",
    ovr.Model("openai/gpt-5.1"),
    ovr.Timeout("30s"),
    ovr.MaxTokens(20000),
    ovr.MaxCostUSD(1.50),
)

```

Rules:

- Duration strings use Go duration syntax, such as `"10s"`, `"2m"`, `"1h"`.
- Token and cost limits must be greater than zero.
- Budgets are inherited by governed child work where applicable.

Retries

```

ovr.Pipe("Load data and summarize it.",
    ovr.Model("openai/gpt-5.1"),
    ovr.Retry(2, ovr.ExponentialBackoff()),
)

```

Retries apply to transient provider failures and retry-safe tool work. Read-only and idempotent tools are retry-safe. Non-idempotent side-effecting tools are not retried as if they were safe.

Worker Pools

Use `WorkerPool` on `From`:

```
ovr.From("POST /tickets", ovr.WorkerPool(20))
```

It bounds concurrent trigger executions. `WorkerPool(0)` is invalid.

Idempotency Keys

Use `IdempotencyKey` on `From`:

```
ovr.From("POST /tickets", ovr.IdempotencyKey("Idempotency-Key"))
```

The runtime uses the named header to reserve duplicate deliveries in the state store.

Signature Verification

Use `VerifySignature` on `From`:

```
ovr.From(ovr.Webhook("stripe"),  
        ovr.VerifySignature("STRIPE_WEBHOOK_SECRET", "Stripe-Signature"),  
        )
```

The first argument is the environment variable containing the shared secret. The second argument is the request header containing the signature.

Cache And Tool Ordering

Disable provider prompt-cache hints for one pipe:

```
ovr.NoCache()
```

Force tool calls from one provider turn to run one at a time:

```
ovr.SequentialTools()
```

Use `SequentialTools` when tool order matters or when parallel tool calls would compete for the same resource.

18. Hooks, Tracing, And State Syntax

These are advanced syntax surfaces. You do not need them for a simple worker.

Hooks

```
hooks := ovr.NewHooks()

err := hooks.Register(ovr.EventToolCallCompleted,
    func(ctx context.Context, event ovr.Event) (ovr.Event, error) {
        event.Payload["observed"] = true
        return event, nil
    })
if err != nil {
    log.Fatal(err)
}

runner := ovr.NewRunner(ovr.WithHooks(hooks))
```

Event kinds include:

```
ovr.EventPipelineStarted
ovr.EventPipelineCompleted
ovr.EventPipelineFailed
ovr.EventPipeStarted
ovr.EventPipeCompleted
ovr.EventPipeFailed
ovr.EventSessionStarted
ovr.EventSessionSaved
ovr.EventSessionCancelled
ovr.EventLLMCallStarted
ovr.EventLLMCallCompleted
ovr.EventLLMCallFailed
ovr.EventToolCallStarted
ovr.EventToolCallCompleted
ovr.EventToolCallFailed
ovr.EventPermissionDecision
ovr.EventIdempotencyDecision
ovr.EventSignatureDecision
ovr.EventHookFailed
ovr.EventSchemaValidationPassed
ovr.EventSchemaValidationFailed
ovr.EventSchemaRepairStarted
ovr.EventSchemaRepairCompleted
ovr.EventSchemaRepairFailed
ovr.EventBudgetExceeded
ovr.EventTaskStarted
ovr.EventTaskCompleted
ovr.EventTaskFailed
ovr.EventSinkLogged
```

Tracing

Install a tracer:

```
runner := ovr.NewRunner(
    ovr.WithTracer(ovr.NopTracer()),
)
```

Custom tracers implement:

```

type Tracer interface {
    StartSpan(ctx context.Context, name string, attrs map[string]any) (context.Context, Span)
}

type Span interface {
    End()
    SetAttribute(key string, value any)
    RecordError(err error)
}

```

State Store

Advanced users may install a custom state store:

```

runner := ovr.NewRunner(
    ovr.WithStateStore(myStore),
)

```

The public `StateStore` interface includes execution history, sessions, idempotency reservations, events, and schema violations.

Most workers can use the default state backend. Environment variables:

```

OUVRIER_STATE_BACKEND=sqlite
OUVRIER_STATE_PATH=/path/to/state.db

```

The backend may also be `memory` for lightweight development or tests.

19. Complete Keyword Reference

Top-Level Runtime

Syntax	Category	Meaning
<code>ovr.Run(addr, nodes...)</code>	Runtime	Starts the default runner.
<code>ovr.NewRunner(opts...)</code>	Runtime	Creates a configured runner.
<code>runner.Run(addr, nodes...)</code>	Runtime	Starts a configured runner.
<code>ovr.Validate(nodes...)</code>	Runtime	Validates declarations without serving.
<code>ovr.RequireEnv(names...)</code>	Env	Fails if required env vars are missing.

Triggers

Syntax	Category	Meaning
<code>ovr.From(source, opts...)</code>	Trigger	Starts a pipeline.
<code>ovr.Cron(expr)</code>	Trigger source	Scheduled trigger.
<code>ovr.Webhook(value)</code>	Trigger or push target	Inbound provider name or outbound URL.
<code>ovr.Stream(uri)</code>	Trigger source	Stream subscriber.
<code>ovr.WorkerPool(limit)</code>	From option	Bounds concurrent trigger work.
<code>ovr.IdempotencyKey(header)</code>	From option	Header-based duplicate prevention.
<code>ovr.VerifySignature(env, header)</code>	From option	HMAC-SHA256 verification.

Pipes

Syntax	Category	Meaning
<code>ovr.Pipe(goal, opts...)</code>	Step	One agent step.
<code>ovr.Model(id)</code>	Pipe option	Required model selector.
<code>ovr.Output[T]()</code>	Pipe option	Typed JSON result contract.
<code>ovr.Timeout(value)</code>	Pipe option	Wall-clock budget.
<code>ovr.MaxTokens(max)</code>	Pipe option	Token budget.
<code>ovr.MaxCostUSD(max)</code>	Pipe option	Cost budget.
<code>ovr.Retry(max, policy?)</code>	Pipe option	Transient retries.
<code>ovr.ExponentialBackoff()</code>	Retry option	Exponential backoff policy.
<code>ovr.NoCache()</code>	Pipe option	Disables prompt-cache hints.
<code>ovr.SequentialTools()</code>	Pipe option	Serializes tool calls in a provider turn.

Tools

Syntax	Category	Meaning
<code>ovr.Tool(name, fn, opts...)</code>	Pipe option	Registers a Go tool.
<code>ovr.ReadOnly()</code>	Tool option	Tool has no side effects.
<code>ovr.Idempotent(keyExpr)</code>	Tool option	Replay-safe side effect.
<code>ovr.SideEffecting(labels...)</code>	Tool option	Non-idempotent side effect.
<code>ovr.RequiresApproval()</code>	Tool option	Requires explicit approval policy.
<code>ovr.Describe(text)</code>	Tool option	Tool description.
<code>ovr.Param(name, description)</code>	Tool option	Parameter description.
<code>ovr.ToolTimeout(value)</code>	Tool option	Per-tool timeout.

Terminals

Syntax	Category	Meaning
<code>ovr.Reply(format)</code>	Terminal	Replies to HTTP caller.
<code>ovr.JSON[T]()</code>	Reply format	Typed JSON reply.
<code>ovr.SSE()</code>	Reply format	Server-sent events reply.
<code>ovr.Accepted()</code>	Reply format	Async HTTP 202 reply.
<code>ovr.Push(target)</code>	Terminal	Sends output elsewhere.
<code>ovr.Queue(uri)</code>	Push target	HTTP or NATS queue target.
<code>ovr.Sink(target)</code>	Terminal	Consumes output without reply.
<code>ovr.Log()</code>	Sink target	Redacted log sink.
<code>ovr.File(path)</code>	Sink target	File sink inside sandbox.

Composition

Syntax	Category	Meaning
<code>ovr.Parallel(items...)</code>	Step	Fan out same input to branches.
<code>ovr.Map(items...)</code>	Step	Apply pipe pipeline to each array item.
<code>ovr.Concurrency(limit)</code>	Map option	Bounds map item concurrency.
<code>ovr.PartialOK()</code>	Parallel, Map, SubAgent option	Allows ordered partial failures.
<code>ovr.Pipeline(nodes...)</code>	SubAgent helper	Builds a child pipe-only pipeline.
<code>ovr.SubAgent(name, pipeline, opts...)</code>	Pipe option	Registers child pipeline as a tool.
<code>ovr.MaxParallel(limit)</code>	SubAgent option	Bounds concurrent subagent calls.

Capabilities

Syntax	Category	Meaning
<code>ovr.Skill(name)</code>	Pipe option	Loads a skill directory.
<code>ovr.MCP(name)</code>	Pipe option	Attaches a configured MCP server.
<code>ovr.Bash(sandbox, opts...)</code>	Pipe option	Adds a sandboxed Bash tool.
<code>ovr.BashTimeout(value)</code>	Bash option	Per-command timeout.
<code>ovr.BashMaxOutputBytes(max)</code>	Bash option	Output capture bound.
<code>ovr.UnsafeBashHostExecution()</code>	Bash option	Host-shell fallback.
<code>ovr.Sandbox(root, opts...)</code>	Runner/Bash helper	Filesystem boundary.
<code>ovr.AllowEnv(keys...)</code>	Sandbox option	Environment allowlist.

Permissions

Syntax	Category	Meaning
<code>ovr.AllowSideEffects(labels...)</code>	Permission policy	Allows non-targeted side-effect labels.
<code>ovr.AllowSideEffectTargets(label, targets...)</code>	Permission policy	Allows target-scoped side effects.
<code>ovr.WithPermissionPolicy(policy)</code>	Runner option	Installs a policy.

Advanced Runner

Syntax	Category	Meaning
<code>ovr.WithStateStore(store)</code>	Runner option	Custom state store.
<code>ovr.WithHooks(hooks)</code>	Runner option	Lifecycle hooks.
<code>ovr.NewHooks()</code>	Hooks	Creates a hook registry.
<code>hooks.Register(kind, hook)</code>	Hooks	Registers a hook.
<code>ovr.WithSandbox(config)</code>	Runner option	Sets default sandbox.
<code>ovr.WithSchemaRepairAttempts(n)</code>	Runner option	Bounded schema repair.
<code>ovr.WithTracer(tracer)</code>	Runner option	Tracer integration.
<code>ovr.NopTracer()</code>	Tracing	No-op tracer.

20. Worked Examples From Simple To Complex

This section is a progression. Each example adds exactly one or two new ideas. Read the examples as architecture patterns, not as isolated snippets to copy blindly.

The progression is:

1. One trigger, one pipe, one reply.
2. Add a typed JSON contract.
3. Add a read-only tool and path parameters.
4. Split a workflow into sequential agent steps.
5. Process a batch with `Map`.
6. Run independent reviewers with `Parallel`.
7. Compose a governed incident-response orchestrator with tools, side effects, a subagent, budgets, retries, and typed final output.

Example 1: One Trigger, One Pipe, One Reply

This is the minimum useful HTTP worker. It has no typed output and no tools. The only Ouvrier structure is:

```
From -> Pipe -> Reply
```

Use this shape when you are still validating a route, a provider key, or a basic prompt.

```
package main

import (
    "log"

    ovr "github.com/ArnaudGuiovanna/ouvrier"
)

func main() {
    if err := ovr.RequireEnv("OPENAI_API_KEY"); err != nil {
        log.Fatal(err)
    }

    if err := ovr.Run(":8080",
        ovr.From("POST /test"),
        ovr.Pipe("Say hello in one short sentence.",
            ovr.Model("openai/gpt-5.1"),
        ),
        ovr.Reply(ovr.SSE()),
    ); err != nil {
        log.Fatal(err)
    }
}
```

Test:

```
curl -N -X POST http://localhost:8080/test -H 'Content-Type: application/json' -d '{"message":"user cannot sign in"}'
```

What this example teaches:

- `From("POST /test")` creates an HTTP route.
- `Pipe(...)` runs one model call.
- `Reply(SSE())` sends the final result as server-sent events.
- The model can only return text here. There is no schema contract yet.

Example 2: Typed JSON Contract

The next step is to make the output shape explicit. `Output[T]()` tells the pipe to produce JSON matching a Go type. `Reply(JSON[T]())` tells the HTTP reply to use that same type.

```
package main

import (
    "log"

    ovr "github.com/ArnaudGuiovanna/ouvrier"
)

type Greeting struct {
    Salutation string `json:"salutation"`
}

func main() {
    if err := ovr.RequireEnv("OPENAI_API_KEY"); err != nil {
```

```

        log.Fatal(err)
    }

    if err := ovr.Run(":8080",
        ovr.From("POST /greeting"),
        ovr.Pipe("Write one friendly greeting. Return JSON only.",
            ovr.Model("openai/gpt-5.1"),
            ovr.Output[Greeting](),
        ),
        ovr.Reply(ovr.JSON[Greeting]()),
    ); err != nil {
        log.Fatal(err)
    }
}

```

What this example adds:

- A Go struct becomes the public response contract.
- JSON tags are part of the API. `json:"salutation"` is the field name the caller sees.
- The model is instructed and validated against the schema.
- If the model returns prose instead of JSON, the pipeline fails unless schema repair is enabled on the runner.

This is the default shape for a production HTTP endpoint:

```
From -> Pipe(Output[T]) -> Reply(JSON[T])
```

Example 3: Read-Only Tool And Path Parameters

A model does not automatically know your application data. Tools expose your domain functions to the agent. This example loads a support ticket by ID.

The route contains `{id}`. Ouvrier wraps path parameters into the first pipe input. The tool still receives structured arguments chosen by the model.

```

package main

import (
    "context"
    "log"

    ovr "github.com/ArnaudGuioivanna/ouvrier"
)

type Ticket struct {
    ID      string `json:"id"`
    Subject string `json:"subject"`
    Body    string `json:"body"`
}

type TicketSummary struct {
    ID        string `json:"id"`
    Priority   string `json:"priority"`
    Summary   string `json:"summary"`
}

type LoadTicketArgs struct {

```

```

    ID string `json:"id"`
}

func LoadTicket(ctx context.Context, args LoadTicketArgs) (Ticket, error) {
    return Ticket{
        ID:      args.ID,
        Subject: "Login issue",
        Body:    "User cannot sign in.",
    }, nil
}

func main() {
    if err := ovr.RequireEnv("OPENAI_API_KEY"); err != nil {
        log.Fatal(err)
    }

    if err := ovr.Run(":8080",
        ovr.From("POST /tickets/{id}"),
        ovr.Pipe("Load the ticket by ID and summarize the support problem.",
            ovr.Model("openai/gpt-5.1"),
            ovr.Tool("load_ticket", LoadTicket,
                ovr.ReadOnly(),
                ovr.Describe("Load one support ticket by ID."),
                ovr.Param("id", "Ticket identifier from path_params.id."),
            ),
            ovr.Output[TicketSummary](),
        ),
        ovr.Reply(ovr.JSON[TicketSummary]()),
    ); err != nil {
        log.Fatal(err)
    }
}

```

What this example adds:

- `Tool(...)` registers a normal Go function.
- `ReadOnly()` tells Ouvrier the tool has no side effects.
- `Describe(...)` and `Param(...)` improve the tool schema shown to the model.
- The model decides when to call `load_ticket`; Ouvrier governs the execution.

Example 4: Sequential Agent Middleware

Use multiple pipes when each stage has a distinct contract. This is agentic middleware: one step normalizes data, the next step reasons over the normalized shape, and the final step writes the caller-facing result.

```

type TicketFacts struct {
    TicketID string `json:"ticket_id"`
    Customer string `json:"customer"`
    Topics   []string `json:"topics"`
}

type RiskProfile struct {
    TicketID string `json:"ticket_id"`
    Severity string `json:"severity"`
    Reason   string `json:"reason"`
}

type SupportReply struct {

```

```

    TicketID string `json:"ticket_id"`
    Subject  string `json:"subject"`
    Body     string `json:"body"`
}

err := ovr.Run(":8080",
    ovr.From("POST /tickets/{id}"),
    ovr.Pipe("Extract stable ticket facts. Return JSON only.",
        ovr.Model("openai/gpt-5.1"),
        ovr.Output[TicketFacts](),
    ),
    ovr.Pipe("Assess operational risk from the ticket facts. Return JSON only.",
        ovr.Model("openai/gpt-5.1"),
        ovr.Output[RiskProfile](),
    ),
    ovr.Pipe("Write the final support reply from the risk profile. Return JSON only.",
        ovr.Model("openai/gpt-5.1"),
        ovr.Output[SupportReply](),
    ),
    ovr.Reply(ovr.JSON[SupportReply]()),
)

```

What this example adds:

- Intermediate outputs are not sent to the caller.
- Each pipe receives the previous pipe's output.
- Different stages can use different types.
- The final `Reply(JSON[SupportReply]())` sends only the last pipe result.

Use this pattern when you want reliable boundaries between extraction, classification, enrichment, drafting, and final formatting.

Example 5: Batch Processing With Map

Use `Map` when one pipe produces an array and the next operation should run for each item. This example extracts incoming tickets from one request, scores each ticket concurrently, and then wraps the ordered scores in a final reply.

```

type IncomingTicket struct {
    ID      string `json:"id"`
    Subject string `json:"subject"`
    Body    string `json:"body"`
}

type TicketScore struct {
    ID      string `json:"id"`
    Priority string `json:"priority"`
    Reason  string `json:"reason"`
}

type BatchScores struct {
    Scores []TicketScore `json:"scores"`
}

err := ovr.Run(":8080",
    ovr.From("POST /ticket-batches"),
    ovr.Pipe("Extract the request body as a JSON array of tickets.",
        ovr.Model("openai/gpt-5.1"),

```

```

        ovr.Output[[]IncomingTicket](),
    ),
    ovr.Map(
        ovr.Concurrency(4),
        ovr.Pipe("Score one ticket for support priority. Return JSON only.",
            ovr.Model("openai/gpt-5.1"),
            ovr.Output[TicketScore](),
        ),
    ),
    ovr.Pipe("Wrap the ordered ticket scores in the final response object.",
        ovr.Model("openai/gpt-5.1"),
        ovr.Output[BatchScores](),
    ),
    ovr.Reply(ovr.JSON[BatchScores]()),
)

```

What this example adds:

- `Output[[]IncomingTicket]()` creates the array required by `Map`.
- `Map(...)` applies its pipe body once per array item.
- `Concurrency(4)` limits parallel item processing.
- Results remain ordered by input item.
- A final pipe can reshape map output into a stable response contract.

Use `PartialOK()` inside `Map` when a best-effort batch is better than failing the whole request:

```

ovr.Map(
    ovr.Concurrency(4),
    ovr.PartialOK(),
    ovr.Pipe("Score one ticket.", ovr.Model("openai/gpt-5.1")),
)

```

Example 6: Independent Review With `Parallel`

Use `Parallel` when several independent branches should inspect the same input. This example drafts a response, then runs three independent reviewers: quality, policy, and customer-impact review.

```

type DraftReply struct {
    Subject string `json:"subject"`
    Body    string `json:"body"`
}

type ReviewSummary struct {
    Approved bool    `json:"approved"`
    Risks    []string `json:"risks"`
    Revision string  `json:"revision"`
}

err := ovr.Run(":8080",
    ovr.From("POST /tickets/{id}"),
    ovr.Pipe("Draft a support reply from the request.",
        ovr.Model("openai/gpt-5.1"),
        ovr.Output[DraftReply](),
    ),
    ovr.Parallel(
        ovr.Pipe("Review the draft for factual clarity.",

```

```

        ovr.Model("openai/gpt-5.1"),
    ),
    ovr.Pipe("Review the draft for policy and safety concerns.",
        ovr.Model("openai/gpt-5.1"),
    ),
    ovr.Pipe("Review the draft for likely customer impact.",
        ovr.Model("openai/gpt-5.1"),
    ),
    ovr.PartialOK(),
),
ovr.Pipe("Merge the ordered reviews and produce a final decision.",
    ovr.Model("openai/gpt-5.1"),
    ovr.Output[ReviewSummary](),
),
ovr.Reply(ovr.JSON[ReviewSummary]()),
)

```

What this example adds:

- `Parallel(...)` fans out the same draft to each branch.
- Branches are declared as `Pipe` nodes.
- Results are ordered exactly like the branch declarations.
- `PartialOK()` means the merge pipe can still receive successful branch results if one reviewer fails.
- A merge pipe should explain how to interpret partial outcomes.

Use `Parallel` for independent checks. Do not use it when branch B depends on branch A; that is a sequential pipe chain.

Example 7: Governed Incident Orchestrator

This is a larger middleware. It is still just Ouvrier syntax, but it combines the major composition tools:

```

HTTP trigger with concurrency and idempotency
-> read-only domain loading
-> parallel risk analysis
-> action extraction
-> map each action through runbook enrichment
-> final coordinator with subagent, idempotent ticket creation, notification
-> typed JSON reply

```

The domain is incident response. The worker receives an incident ID, loads the current incident, runs independent analysis branches, enriches actions with runbook data, optionally creates an escalation ticket, optionally notifies the on-call channel, and returns the decision.

The important syntax idea is that every risky capability is explicit:

- `load_incident` and `search_runbook` are `ReadOnly()`.
- `open_escalation` is `Idempotent("incident_id")`.
- `notify_on_call` is `SideEffecting("pager")`.
- The runner allows only the `pager` side-effect label.
- The subagent is declared as a child `Pipeline` and exposed with `SubAgent`.

```

package main

import (
    "context"
    "log"

    ovr "github.com/ArnaudGuiovanna/ouvrier"
)

type Incident struct {
    ID          string `json:"id"`
    Service     string `json:"service"`
    Summary     string `json:"summary"`
    Signals     []string `json:"signals"`
    CustomerIDs []string `json:"customer_ids"`
}

type IncidentContext struct {
    IncidentID string `json:"incident_id"`
    Service    string `json:"service"`
    Summary    string `json:"summary"`
    Signals    []string `json:"signals"`
}

type RiskAssessment struct {
    Severity string `json:"severity"`
    Risks    []string `json:"risks"`
}

type CustomerImpact struct {
    ImpactedCustomers []string `json:"impacted_customers"`
    ImpactSummary    string  `json:"impact_summary"`
}

type ActionDraft struct {
    IncidentID string `json:"incident_id"`
    Action     string `json:"action"`
    Why        string `json:"why"`
}

type RunbookMatch struct {
    Title string `json:"title"`
    URL   string `json:"url"`
    Step  string `json:"step"`
}

type ActionPlan struct {
    IncidentID string `json:"incident_id"`
    Action     string `json:"action"`
    RunbookURL string `json:"runbook_url"`
    Owner      string `json:"owner"`
    Rollback   string `json:"rollback"`
}

type RemediationDraft struct {
    Steps []string `json:"steps"`
    Risks []string `json:"risks"`
}

type RemediationPlan struct {
    Steps        []string `json:"steps"`
    RollbackPlan string `json:"rollback_plan"`
    Confidence   string `json:"confidence"`
}

```

```

type EscalationTicket struct {
    ID string `json:"id"`
    URL string `json:"url"`
}

type NotificationResult struct {
    Channel string `json:"channel"`
    Sent bool `json:"sent"`
}

type IncidentDecision struct {
    IncidentID string `json:"incident_id"`
    Severity string `json:"severity"`
    Actions []ActionPlan `json:"actions"`
    Remediation RemediationPlan `json:"remediation"`
    EscalationTicket *EscalationTicket `json:"escalation_ticket,omitempty"`
    Notification *NotificationResult `json:"notification,omitempty"`
    OperatorSummary string `json:"operator_summary"`
}

type LoadIncidentArgs struct {
    ID string `json:"id"`
}

func LoadIncident(ctx context.Context, args LoadIncidentArgs) (Incident, error) {
    return Incident{
        ID: args.ID,
        Service: "auth-api",
        Summary: "Elevated login failures in one region.",
        Signals: []string{
            "5xx rate above baseline",
            "password login failure spike",
            "region=eu-west",
        },
        CustomerIDs: []string{"cust_123", "cust_456"},
    }, nil
}

type SearchRunbookArgs struct {
    Service string `json:"service"`
    Query string `json:"query"`
}

func SearchRunbook(ctx context.Context, args SearchRunbookArgs) (RunbookMatch, error) {
    return RunbookMatch{
        Title: "Auth API regional login incident",
        URL: "https://runbooks.example.com/auth-api/login-region",
        Step: "Check regional dependency health before rollback.",
    }, nil
}

type OpenEscalationArgs struct {
    IncidentID string `json:"incident_id"`
    Severity string `json:"severity"`
    Summary string `json:"summary"`
}

func OpenEscalation(ctx context.Context, args OpenEscalationArgs) (EscalationTicket, error) {
    return EscalationTicket{
        ID: "ESC-" + args.IncidentID,
        URL: "https://incidents.example.com/ESC-" + args.IncidentID,
    }, nil
}

```

```

type NotifyOnCallArgs struct {
    IncidentID string `json:"incident_id"`
    Channel    string `json:"channel"`
    Message    string `json:"message"`
}

func NotifyOnCall(ctx context.Context, args NotifyOnCallArgs) (NotificationResult, error) {
    return NotificationResult{Channel: args.Channel, Sent: true}, nil
}

func main() {
    if err := ovr.RequireEnv("OPENAI_API_KEY"); err != nil {
        log.Fatal(err)
    }

    remediationPlanner := ovr.Pipeline(
        ovr.Pipe("Draft a remediation plan from the incident context and actions.",
            ovr.Model("openai/gpt-5.1"),
            ovr.Output[RemediationDraft](),
        ),
        ovr.Pipe("Critique the draft, add rollback steps, and return the final plan.",
            ovr.Model("openai/gpt-5.1"),
            ovr.Output[RemediationPlan](),
        ),
    )

    runner := ovr.NewRunner(
        ovr.WithSchemaRepairAttempts(1),
        ovr.WithPermissionPolicy(
            ovr.AllowSideEffects("pager"),
        ),
    )

    if err := runner.Run(":8080",
        ovr.From("POST /incidents/{id}",
            ovr.WorkerPool(10),
            ovr.IdempotencyKey("Idempotency-Key"),
        ),
        ovr.Pipe("Load the incident by path_params.id and normalize its context.",
            ovr.Model("openai/gpt-5.1"),
            ovr.Tool("load_incident", LoadIncident,
                ovr.ReadOnly(),
                ovr.Describe("Load one incident by ID."),
                ovr.Param("id", "Incident ID from path_params.id."),
            ),
            ovr.Output[IncidentContext](),
        ),
        ovr.Parallel(
            ovr.Pipe("Assess operational risk. Return JSON only.",
                ovr.Model("openai/gpt-5.1"),
                ovr.Output[RiskAssessment](),
            ),
            ovr.Pipe("Assess customer impact. Return JSON only.",
                ovr.Model("openai/gpt-5.1"),
                ovr.Output[CustomerImpact](),
            ),
        ),
        ovr.PartialOK(),
    ),
        ovr.Pipe("Merge the analyses into an ordered array of concrete actions.",
            ovr.Model("openai/gpt-5.1"),
            ovr.Output[[]ActionDraft](),
        ),
    ),
    ovr.Map(

```

```

        ovr.Concurrency(3),
        ovr.Pipe("Attach owner, runbook evidence, and rollback notes to one action.",
            ovr.Model("openai/gpt-5.1"),
            ovr.Tool("search_runbook", SearchRunbook,
                ovr.ReadOnly(),
                ovr.Describe("Find a runbook entry for a service and query."),
                ovr.Param("service", "Service name."),
                ovr.Param("query", "Incident or action search query."),
            ),
            ovr.Output[ActionPlan](),
        ),
    ),
    ovr.Pipe("Coordinate the final incident decision. Use remediation_planner for the final plan.
Open an escalation for high severity incidents. Notify on-call only for immediate action.",
        ovr.Model("openai/gpt-5.1"),
        ovr.SubAgent("remediation_planner", remediationPlanner,
            ovr.MaxParallel(2),
        ),
        ovr.Tool("open_escalation", OpenEscalation,
            ovr.Idempotent("incident_id"),
            ovr.Describe("Create or reuse one escalation ticket for an incident."),
            ovr.Param("incident_id", "Incident identifier."),
            ovr.Param("severity", "Incident severity."),
            ovr.Param("summary", "Escalation summary."),
        ),
        ovr.Tool("notify_on_call", NotifyOnCall,
            ovr.SideEffecting("pager"),
            ovr.Describe("Send a message to the on-call channel."),
            ovr.Param("incident_id", "Incident identifier."),
            ovr.Param("channel", "On-call channel name."),
            ovr.Param("message", "Notification text."),
        ),
        ovr.Timeout("45s"),
        ovr.Retry(1, ovr.ExponentialBackoff()),
        ovr.Output[IncidentDecision](),
    ),
    ovr.Reply(ovr.JSON[IncidentDecision]()),
); err != nil {
    log.Fatal(err)
}
}

```

Read the final example in layers:

```

Layer 1: From("POST /incidents/{id}") accepts incident work.
Layer 2: load_incident gives the model trusted incident data.
Layer 3: Parallel runs risk and customer-impact analysis at the same time.
Layer 4: a merge pipe turns analysis results into []ActionDraft.
Layer 5: Map enriches each action with runbook data under Concurrency(3).
Layer 6: remediation_planner runs a child two-pipe planning workflow.
Layer 7: open_escalation and notify_on_call are real side effects.
Layer 8: IncidentDecision is the only HTTP response contract.

```

Why this is materially more complex:

- There are multiple typed contracts, not one final schema.
- The pipeline mixes sequential, parallel, and per-item execution.
- It uses two different classes of tools: read-only data access and governed side effects.

- The idempotent tool can be retried safely by key; the pager notification needs an explicit permission label.
- A subagent encapsulates a reusable child workflow instead of hiding logic in one large prompt.
- `PartialOK()` makes the parallel analysis resilient while keeping result order deterministic.
- `Timeout`, `Retry`, and `WithSchemaRepairAttempts` make failure behavior explicit.

This pattern is appropriate when Ouvrier is acting as middleware between incoming operational events, internal data sources, action systems, and an HTTP client that needs a concise final decision.

21. Validation And Troubleshooting Checklist

Use this checklist when a declaration does not work.

Structure

- Did the pipeline start with `ovr.From(...)`?
- Does every top-level pipeline end with exactly one `Reply`, `Push`, or `Sink`?
- Is the terminal last?
- Are multiple top-level pipelines separated by a new `From`?
- Are `Parallel`, `Map`, and subagent `Pipeline` bodies made only of `Pipe` nodes where required?

Models And Providers

- Does every `Pipe` include `ovr.Model(...)`?
- Does the model use `provider/model` form?
- Does the provider prefix match the environment variable you configured?
- Did the process actually load `.env` before `go run .`?
- Are you seeing `provider_not_configured` because the model prefix is wrong?

Typed Output

- Are all JSON fields exported Go fields?
- Are JSON tags spelled correctly?
- If both `Output[T]()` and `Reply(JSON[U]())` are present, are `T` and `U` the same type?
- Does the model goal say to return JSON only?
- Should you enable `WithSchemaRepairAttempts(1)` for a stricter workflow?

Tools

- Does the tool function start with `context.Context`?
- Does it return `error` or `(value, error)`?
- Did you classify it as `ReadOnly`, `Idempotent`, or `SideEffecting`?

- Did you add `Describe` and `Param` text so the model knows how to call it?
- If the tool mutates state, did you install a permission policy?

Side Effects

- Remember: a model cannot perform real side effects from text instructions.
- File writes need a Go tool or `Sink(File(...))`.
- Webhook and queue pushes need `AllowSideEffectTargets`.
- File sinks need `withSandbox` and `AllowSideEffectTargets("file", path)`.
- Bash needs sandbox syntax and process/filesystem target permissions.

Composition

- Does `Map` receive a JSON array?
- Is `Concurrency(limit)` greater than zero?
- Is `MaxParallel(limit)` between 1 and 5?
- Do you need `PartialOK()` for best-effort batches?
- Are you using a `SubAgent` only when a reusable child pipeline is clearer than a single tool?

Trigger Compatibility

- `Reply` works only with HTTP triggers.
- Cron and stream triggers should use `Push` or `Sink`.
- Webhook trigger names are provider names, not URLs.
- Webhook push targets are URLs.
- `VerifySignature` names an env var and a header.
- `IdempotencyKey` names a header.

Good Defaults

- Start with one `From`, one `Pipe`, one `Reply(JSON[T]())`.
- Add `Output[T]()` as soon as the response shape matters.
- Add tools only when the model needs real data or real side effects.
- Mark tools `ReadOnly()` by default when they only fetch data.
- Use `NewRunner` only when you need permissions, hooks, tracing, sandbox, schema repair, or custom state.
- Prefer clear, narrow pipe goals over large vague prompts.
- Validate early with `ovr.Validate(...)` when composing complex graphs.